Senior Ontology - Smart Route & User Management API

Unit test specification: Route scoring — No preference (3.0) vs With preference (4.0)

Scope note: DFD shows **Google Map API** for route list, then DSS scoring with **Pollution** and **Relative Humidity** data. These tests run against **POST /scoreRoutes** in **server.py** with mocked HTTP clients. **3.1–3.2 / 4.1–4.3** (get input, generate list via Maps) are implemented in the **Flutter** app; here we supply encoded polyline + distance/duration as the backend input after routes exist. **3.3–3.4 / 4.4–4.5** (selected route + detail) map to scoring one route and returning di, dt, dp, dw, risk_score.

Test suite C — Process 3.0 No pollution preference

Test Case ID	TC-ROUTE-3.0
Python module	UnitTest_ScorePollutant.py
Key test	test_context_3_find_route_no_pollution_preference
API	POST /scoreRoutes, focus_pollutants = None

3.1 Get route input — Represented by request fields: encoded polyline, distance_meters, duration_seconds.
3.2 Generate route list — Not in server unit test (Directions API in app).
3.3 Get selected route — One route id in request exercises single-route scoring.
3.4 Generate detail of route — Response includes risk_score, di, dt, dp, dw, humidity fields.

Step	Action	Expected Result	Status
1	scoreRoutes without focus_pollutants	HTTP 200, scores length = routes	Pass
2	Inspect risk_score and DSS fields	Values in valid range	Pass
3	Critic weights sum to 1	No user pollutant boost	Pass

Test suite D — Process 4.0 With pollution preference

Test Case ID	TC-ROUTE-4.0
Python module	UnitTest_ScorePollutant.py
Key test	test_context_4_find_route_with_pollution_preference
API	POST /scoreRoutes, focus_pollutants e.g. [pm2.5]

4.1 Get route input — Same as 3.1.
4.2 Get pollution input — **focus_pollutants** list (maps to Preferred Pollution / ontology selection in app).
4.3 Generate route list — Maps still in app; server re-weights critic before pollutant score.
4.4–4.5 — Scoring detail reflects boosted weight on focused pollutant.

Step	Action	Expected Result	Status
1	scoreRoutes baseline then with focus pm2.5	Second run higher pm2.5 critic weight	Pass
2	Mock Pollution + Humidity APIs	Deterministic DSS inputs	Pass

Supporting unit tests (API integration pieces)

DFD / requirement	Unit test method
Pollution API handling	test_fetch_aqi_*, test_extract_pollutants_*
Humidity API handling	test_fetch_humidity_branches
DSS math	test_route_pollution_score_*, test_evaluate_route_all_branches, test_critic_weights_*
Ontology helpers (optional path)	test_ontology_*, test_get_ontology_maps_*

Coverage: run **backend_tests/run_coverage.sh** — UnitTest_ScorePollutant.py 100%. Generated 2026-04-06 10:40.